HumanEvo: An Evolution-aware Benchmark for More Realistic Evaluation of Repository-level Code Generation

Dewu Zheng[1], Yanlin Wang[1*], Ensheng Shi[2], Ruikai Zhang[3], Yuchi Ma[3], Hongyu Zhang[4], Zibin Zheng[1,5]
[1] Sun Yat-sen University, Zhuhai, China
zhengdw5@mail2.sysu.edu.cn, {wangylin36, zhzibin}@mail.sysu.edu.cn,
[2] Huawei Cloud Computing Technologies Co., Ltd., Beijng, China
shiensheng@huawei.com,
[3] Huawei Cloud Computing Technologies Co., Ltd., Shenzhen, China
{zhangruikai1, mayuchi1}@huawei.com
[4] Chongqing University
hyzhang@cqu.edu.cn
[5] Zhuhai Key Laboratory of Trusted Large Language Models

2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE) | 979-8-3315-0569-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICSE55347.2025.00228

***Abstract*—To evaluate the repository-level code generation capabilities of Large Language Models (LLMs) in complex real-world software development scenarios, many evaluation methods have been developed. These methods typically leverage contextual code from the latest version of a project to assist LLMs in accurately generating the desired function. However, such evaluation methods fail to consider the dynamic evolution of software projects over time, which we refer to as evolution-ignored settings. This in turn results in inaccurate evaluation of LLMs' performance. In this paper, we conduct an empirical study to deeply understand LLMs' code generation performance within settings that reflect the evolution nature of software development. To achieve this, we first construct an evolution-aware repository-level code generation dataset, namely HumanEvo, equipped with an automated execution-based evaluation tool. Second, we manually categorize HumanEvo according to dependency levels to more comprehensively analyze the model's performance in generating functions with different dependency levels. Third, we conduct extensive experiments on HumanEvo with seven representative and diverse LLMs to verify the effectiveness of the proposed benchmark. We obtain several important findings through our experimental study. For example, we find that previous evolution-ignored evaluation methods result in inflated performance of LLMs, with performance overestimations ranging from 10.0% to 61.1% under different context acquisition methods, compared to the evolution-aware evaluation approach. Based on the findings, we give actionable suggestions for more realistic evaluation of LLMs on code generation. We also build a shared evolution-aware code generation toolbox to facilitate future research. The replication package including source code and datasets is anonymously available at https://github.com/DeepSoftwareAnalytics/HumanEvo.**

I. Introduction

In recent years, the LLM-based code generation task has drawn widespread attention [1]–[10]. Many code LLMs [11]–[23] are being employed to empower programming assistants, which are playing a vital role in practical software development [24]–[27]. Recently, several repository-level code generation benchmarks such as CoderEval [28], RepoBench [29], EvoCodeBench [30], and RepoEval [31] have emerged to better simulate real-world development scenarios. They sample functions from real-world projects as programming tasks, aiming to reflect the performance of LLMs in actual development by prompting them to generate these functions according to the target function description and project context.

Despite the fact that real-world projects are inherently dynamic and evolve over time [32]–[34], current code generation benchmarks often overlook this critical aspect. They tend to treat the latest version of the repository as context source for evaluation, a situation we term as the *evolution-ignored* phenomenon. This evaluation approach fails to accurately capture true nature of real software development scenarios where projects continuously evolve over time, which means that the project context that developers face when writing different code changes constantly. Therefore, to accurately assess the repository-level coding capabilities of LLMs, it is imperative to factor in this evolution aspect. Specifically, when prompting LLMs to generate different target functions, the project context provided should mirror the one available to developers at the time of the target function's creation. Unfortunately, evolution-ignored evaluation in previous benchmarks provides LLMs with an inaccurate and potentially misleading programming environment, leading to serious issues. We have found concrete instances of issues that stem from the evolution-ignored setting in real GitHub projects, which are detailed in Section II.

Benchmark HumanEvo. To fill this gap, we construct HumanEvo, a novel evolution-aware repository-level code generation benchmark, which better simulates real-world development processes and reflects the evolution nature of projects over time. The evaluation process of HumanEvo is as follows: to ensure that the context we provide to LLMs mirrors the context available to the programmer when writing the target

* Yanlin Wang is the corresponding author.

1558-1225/25/$31.00 ©2025 IEEE
DOI 10.1109/ICSE55347.2025.00228

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

function (neither including additional code introduced by subsequent version changes nor omitting code that may have been updated or deleted due to version change), we roll back the entire repository to the state before the target code was committed and evaluate the code generation performance of LLMs at function granularity on the rolled-back repository.

We conduct a rigorous data construction pipeline to construct HumanEvo. We start with selecting high-quality projects and then collecting a large number of pull requests (PRs) from these projects. Then, we perform an initial filtering on the crawled PRs to meet three attributes to guarantee the quality of the newly added functions in PRs. Additionally, to ensure the reliability of the HumanEvo's test suite, we establish a runtime environment for each PR and execute the corresponding project's testing framework to verify that the newly added functions are covered by the test suite. Finally, we obtain 400 task instances, 200 for Python and 200 for Java. For each task instance in HumanEvo, we record its corresponding PR's metadata to roll back the repository to the state prior to the target function's commit.

Empirical Study. Based on the new benchmark HumanEvo, we perform the first study to reveal LLMs' repository-level code generation capability in the evolution-aware setting. In particular, we conduct an extensive evaluation of 7 mainstream LLMs, including both open-source ones (CodeLlama-7B, CodeLlama-13B, CodeLlama-34B [11], DeepSeekCoder-6.7B, DeepSeekCoder-33B [13]) and closed-source ones (GPT-4 [35] and GPT-3.5-Turbo [36]) on HumanEvo.

According to the empirical results, we have the following findings: ① We find that all the studied LLMs show much worse performance on our evolution-aware setting. Previous evolution-ignored evaluation would lead to the inflated performance of LLMs, ranging from 10.0% to 61.1%, revealing how previous evaluation methods provided the models with a deviated programming scenario from reality. ② We find that the impact of the evolution-ignoring setting is more severe for target code with more complex dependencies. Specifically, compared to the evolution-ignored setting, the success rate of LLMs in generating functions with intra-class dependencies under the evolution-aware setting has been observed to decrease by approximately 14.0% on average, while for functions with inter-class dependencies, the decline is 30.9% on average. ③ We investigate how the performance of LLMs change as the repository evolve. Experimental results show that in the evolution-ignored setting, in general, models' performance gradually deviates from the real performance as the project evolves. ④ We find that, under the evolution-ignored setting, utilizing docstrings in different styles as input consistently leads to inflated performance. Experimental results indicate that LLMs' code generation performance improves when given a detailed docstring compared to a brief one.

This work makes the following key contributions:

- We have identified a common flaw in previous repository-level code generation benchmarks: they are evolution-ignored, causing inaccurate simulations of real-world development scenarios.

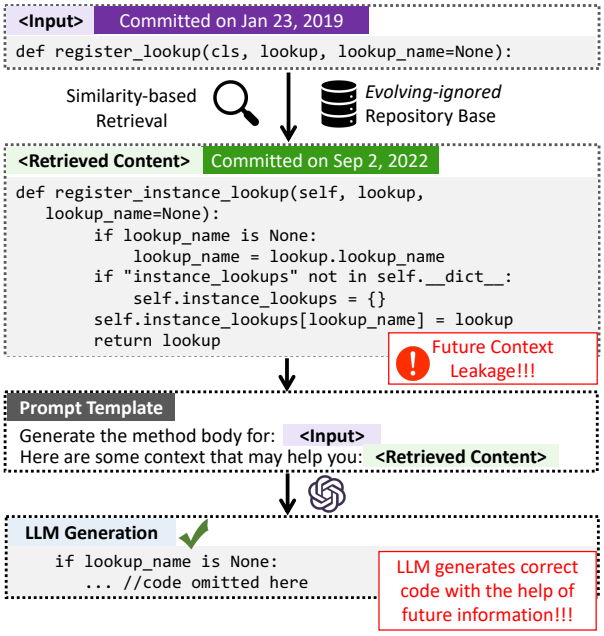


Fig. 1. Example 1: future context leakage.

- We introduce a new benchmark HumanEvo, an evolution-aware repository-level code generation benchmark that addresses the evolution-ignored issue, to conduct a comprehensive empirical study on the performance of LLMs in repository-level code generation.
- Through extensive experimentation, we substantiate the evolution-ignored situation leads to inflated performance of the models, with LLMs showing a performance inflation ranging from 10.0% to 61.1%.
- For convenient usage of HumanEvo, we release all data, code, and the evaluation platform with Docker images that provide runtime environments for all projects, facilitating automated execution of all test suites. The replication package is provided at https://anonymous.4open.science/r/HumanEvo/.

II. Motivating Examples

In this section, we firstly elaborate on how real repository evolves over time. Then, we present two motivating examples[1] that showcase the specific issues that can emerge under the evolution-ignored setting.

In real-world development, projects undergo continuous evolution over time [37]–[39]. During the development phase of a project, programmers need to develop the project (i.e., by adding new features, removing existing code, adding test code, etc) based on the project requirements. In the maintenance phase, programmers continuously monitor the project and promptly fix any bugs that arise. These activities result in varying degrees of modifications to the project, causing the project to evolve over time [40], [41]. For a realistic and

[1]The motivating examples are from the real-world project https://github.com/django/django

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

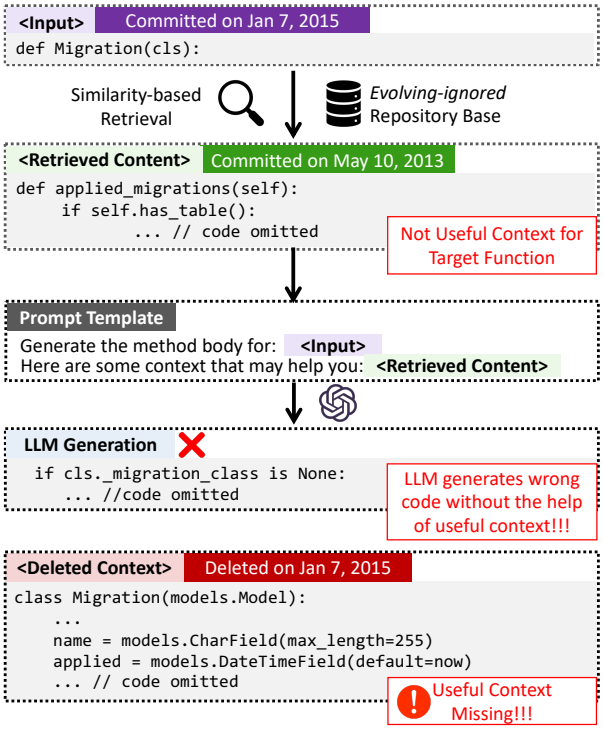
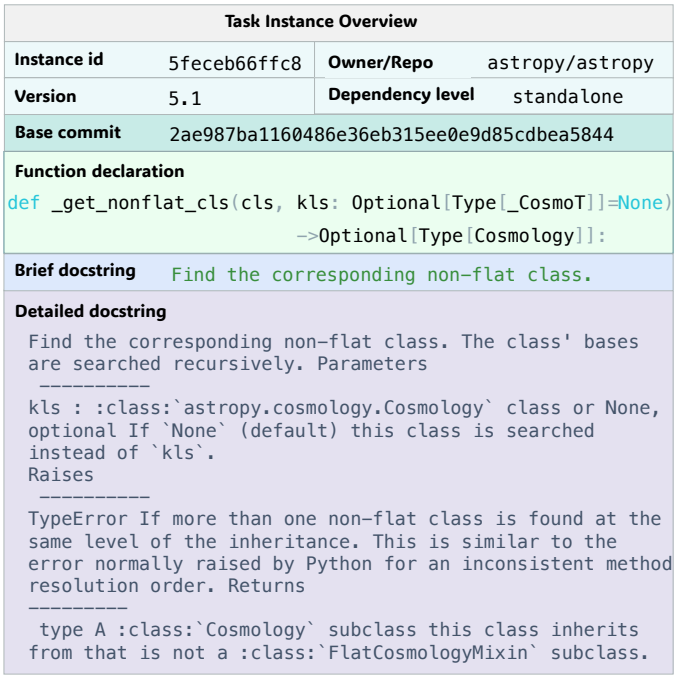


Fig. 2. Example 2: useful context missing.



Task Instance Overview

Instance id	5feceb66ffc8	**Owner/Repo**	astropy/astropy
Version	5.1	**Dependency level**	standalone
Base commit	2ae987ba1160486e36eb315ee0e9d85cdbea5844		

Function declaration

```
def _get_nonflat_cls(cls, kls: Optional[Type[_CosmoT]]=None)
                                  ->Optional[Type[Cosmology]]:
```

Brief docstring Find the corresponding non-flat class.

Detailed docstring

```
Find the corresponding non-flat class. The class' bases
are searched recursively. Parameters
 ----------
kls : :class:`astropy.cosmology.Cosmology` class or None,
optional If `None` (default) this class is searched
instead of `kls`.
Raises
 ----------
TypeError If more than one non-flat class is found at the
same level of the inheritance. This is similar to the
error normally raised by Python for an inconsistent method
resolution order. Returns
 ---------
 type A :class:`Cosmology` subclass this class inherits
from that is not a :class:`FlatCosmologyMixin` subclass.
```

Fig. 3. Task instance overview.

accurate evaluation of LLMs' code generation capabilities, this evolution nature should be considered. Otherwise, severe issues under the evolution-ignored setting would emerge. We have identified concrete instances of issues, which we refer to as *future context leakage* and *use context missing*, which we will illustrate with the following two examples.

Example 1. Figure 1 illustrates the **future context leakage** issue that can arise from the evolution-ignored setting. This example involves a target function that was originally committed on Jan 23, 2019. In the canonical RAG code generation pipeline, a retriever finds similar code from the corresponding repository (which is the latest version in the evolution-ignored setting) to help LLMs generate the target code. However, the retriever yields a function committed on Sep 2, 2022, a date that is *later than the target function*. This process is illogical because it is implausible for a developer to access code that was not yet written at the time of the creation of the target function. Therefore, providing this code snippet as context to the model is a future context leak. While in this example the LLM generates the correct code, the reliability is questionable as it is uncertain whether it can still generate correctly without the help of this future code.

Example 2. Figure 2 illustrates the **useful context missing** issue that may arise in the evolution-ignored setting. In this example, the function committed on Jan 7, 2015 is the target function. In the same RAG code generation pipeline as in Example 1, the retrieved code snippet is not particularly useful and LLM fails to generate the correct code. However, when the developer wrote that code, there was actually a piece of code in the repository that was highly similar to the target code and could have been retrieved as context, which would have likely helped generate the correct code. But this code snippet was deleted during the evolution process. As a result, under the evolution-ignored setting, this piece of code did not exist in repository context. While in this example the LLM generates wrong code, it is possible that providing LLM with this useful context might have enhanced its performance.

III. HumanEvo Benchmark

In this section, we introduce our new benchmark HumanEvo in detail. We present the benchmark overview (Section III-A), benchmark construction pipeline (Section III-B), and benchmark characteristics (Section III-C).

A. Benchmark Overview

HumanEvo consists of a total of 400 task instances, comprising 200 Python programming tasks and 200 Java programming tasks, respectively. Figure 3 shows an example of a task instance overview. Each task instance is associated with a unique identification number (`Instance id`). The `Owner/Repo` field helps identify the GitHub repository from which the selected function originates, while the `Version` field indicates the version of the repository according to the implementation time of this function. `Base commit` refers to the last commit on the branch before the target function was committed. We use it to restore the repository to the state before the target function was implemented. The `Function declaration` of the target function will be provided to the LLM as part of the prompt, along with its corresponding docstring. Since the docstring styles of functions in pragmatic projects vary, with some adopting line-by-line comments and

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

others even lacking corresponding comments, we think it is necessary to rewrite the docstring for each selected function. As depicted in Figure 3, we provide two styles of docstrings. The `Brief docstring` provides a concise summary of the functionality of the target function, while the `Detailed docstring` provides a thorough description of the function's functionality, inputs, and outputs.

B. Benchmark Construction Pipeline

As depicted in Figure 4, the construction pipeline of HumanEvo consists of six steps, namely, (1) Project Selection; (2) Pull Request Crawling; (3) Attribute-based Filtering; (4) Execution-based Filtering; (5) Function Selection; and (6) Human Annotation.

(1) Project Selection. In order to make HumanEvo more representative, we adopt a strict approach to selecting functions from various open-source projects. For Python, we select high-quality real-world projects from a list of the top 5,000 most downloaded PyPI libraries[2]. Subsequently, we deploy the selected projects and run their testing frameworks to ensure each of them can run correctly and stably. We then proceed to collect PRs from these repositories through the GitHub developer API[3]. For Java, as widely recognized projects typically indicate extensive documentation, structured open-source development guidelines, and functional, well-formatted code, our approach involves searching for the top 200 starred projects on GitHub. Then, we deploy every project on the list to select projects that can be successfully deployed and pass the corresponding test suite. Finally, we retain 30 real-world projects, covering over 50 common development domains, which guarantees that the functions selected for evaluation in HumanEvo could represent real-world programming scenarios.

(2) Pull Request Crawling. We crawl a large number of initial PRs from the popular open-source repositories selected in the procedure above. Specifically, for each project, we do not crawl all PRs as we find that early versions of many projects are difficult to deploy successfully. Therefore, for Python, we crawl PRs created after September 2015 (the release time of Python 3.5); for Java, we collect PRs created after September 2014 (the release time of Java 8). Finally, we obtain over 10,000 PRs and meticulously record metadata pertaining to these PRs for subsequent filtering processes.

(3) Attribute-based Filtering. In this step, we preliminarily filter the crawled PRs based on the following three attributes.

- The status of the PR is "Merged". A "Merged" status indicates that the code changes associated with the PR were accepted and incorporated into its parent repository.
- The PR must introduce new functions. These newly added functions are typically intended to develop the software project by adding new features, improvements, or fulfilling general development objectives, which can be selected as programming task.
- The PR must introduce at least one or more new tests, and these tests must cover the selected function.

[2]https://hugovk.github.io/top-pypi-packages/

[3]https://api.github.com

All these attributes aim to ensure that the subsequently selected functions are of high quality, have undergone rigorous review and are covered by the project's testing framework.

(4) Execution-based Filtering. In this step, we perform execution-based filtering for the PRs filtered in the attributes filtering step. Note that, although both the project selection step and the execution-based filtering step have the "Successful installation" criterion, their purposes are different. The main purpose of requiring successful installation in the project selection step is to quickly filter projects, while in this step, it ensures that all versions associated with different PRs can run successfully, as PRs may correspond to multiple versions of a project.

Firstly, we prepare the execution environment for each candidate task instance in HumanEvo. For each task instance, we first determine the version of the project corresponding to the PR, then manually identify the required dependency environment for running the project at that version. For example, for a Python task instance, we confirm the required Python version and the types and versions of third-party libraries it depends on. After obtaining this dependency environment information, we create a virtual execution environment for each version of the project. Task instances that can not install the dependency environment properly are filtered out during the virtual execution environment setup process. After creating the execution environment, we execute the project's testing framework in the corresponding environment.

Secondly, for each selected function, we execute the project's test framework in its required environment to ensure that the selected function is covered by the project's test framework. Specifically, we split the PR's patch file, which is also known as the code diff file, into two parts, namely the production code patch and the test code patch. For each candidate task, we apply the PR's test patch and log the associated test results before and after applying the PR's production patch. Then, we filter out task instances with no test where its status changes from a fail to pass, which indicates the newly added functions are covered by the test suite. After filtering out instances without a fail-to-pass transition, we obtain 400 PRs.

(5) Function Selection. In this step, we manually select functions from the PRs that have passed the filtering process above to serve as programming tasks. After completing the above filtering, we confirm that the remaining PRs meet our requirements. Therefore, we select newly added functions from these PRs as programming tasks. During the selection process, to avoid choosing basic functions, we filter out initialization functions, constructors, and destructors, to ensure the quality of the selected programming tasks. In addition, we ensure that the selected target functions are covered by the project's test framework through manual review . Finally, we acquire a total of 400 programming tasks.

(6) Human Annotation. In this step, we recruit six annotators with more than five years of programming experience to manually rewrite two different styles of docstrings for the selected functions and label each function with a dependency

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

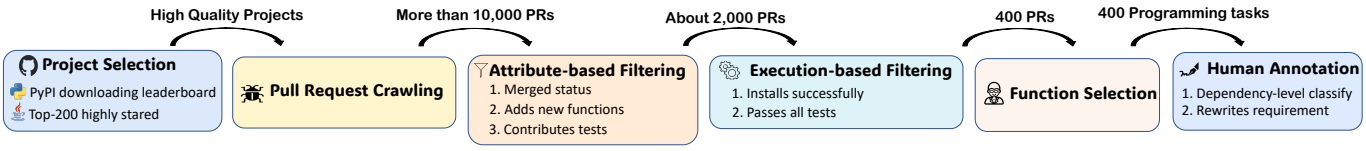


Fig. 4. HumanEvo construction pipeline.

level. We rewrite the docstring for each instance in HumanEvo to ensure the quality of the benchmark dataset and mitigate data leakage issues as much as possible. Specifically, in the process of data construction, we find that the docstring styles of actual projects can be divided into two categories: the detail and the brief. Docstrings are used as input for models, so variations in docstring style may significantly influence model evaluation. Aiming to determine whether the evolution-ignored setting leads to unreal LLMs' performance evaluations across different input styles, we provide two types of the docstring for each task instance. The detailed docstring describes the function of the target function in detail, as well as the types and meanings of its inputs and outputs. In contrast, a brief docstring only briefly describes what the target function does. After the annotators complete writing the docstrings, we conduct a cross-validation review, requiring each task instance to be verified and confirmed by two developers to ensure accuracy. If the annotators disagree with a docstring written by the previous annotator during the review process, they will discuss the issue together until all annotators reach an agreement.

In order to facilitate the evaluation of the ability of LLMs in generating functions with different dependency levels, we further categorize all task instances in HumanEvo into three categories: standalone, intra-class, and inter-class functions. A dependency denotes the invocation of elements defined within projects, and the dependency level is defined based on the position of the target function's dependencies. Standalone functions refer to those that invoke or access only built-in functions and standard libraries. Intra-class functions rely on other functions within the same class. Inter-class functions involve calling functions within the same file or functions in other files.

C. Benchmark Characteristics

*1) **Evolution-aware Benchmark**:* HumanEvo is the pioneering repository-level code generation benchmark to introduce the evolution-aware concept. We highlight a significant oversight in prior research: the developmental trajectory of a project is an evolution process, altering the project context available to programmers as the project advances through various stages. Previous studies have overlooked this temporal dimension, resulting in LLMs being fed with code that had not yet been implemented in the project when certain functions were developed. Consequently, it also misses some context that should have been present but might have been deleted or updated during the project's evolution. This undoubtedly leads to the model facing an unrealistic development scenario. As a result, the performance of LLMs in real-world software development tasks remains unknown. Therefore, we are committed to enhancing HumanEvo to serve as a benchmark that better simulates real-world software code development scenarios, aiming to more accurately reflect the true performance of LLMs applied to pragmatic development tasks.

*2) **Strict Data Filtering Process**:* We ensure the quality of the selected function from several aspects. First, we ensure that the selected projects are of high quality because high-quality projects generally have excellent maintenance, comprehensive testing frameworks, and good coding styles. For Python, we choose the projects with the highest download count in PyPI. For Java, we select projects from the top 200 projects on GitHub based on the number of stars, with an average of around 24k stars. Previous benchmarks have ensured the functionality of target functions by writing unit tests for each function separately. However, we believe that in a project with complex dependencies, adding a new function may have certain impacts on other parts of the project. Relying solely on unit tests for the target function is insufficient to ensure that the project can still function properly and pass the project's test framework after inserting the function. Therefore, in this work, we directly run the project's test framework to ensure the correctness of the target function's functionality while ensuring that inserting the generated function does not have a negative impact on other parts of the project.

In addition, we also verify that our selected functions are covered by the project's test framework through execution-based validation. First, we decompose the code diff in the PR corresponding to the target function into a source code patch and a test code patch, and directly apply the test code patch. Then, we validate whether the target function is covered by the project's test framework by executing the project's test framework twice. In the first execution, we run the project's test framework without applying the source code patch. In the second execution, we apply the source code patch and then run the test framework. If there is a test function in the project's test framework that targets the target function, and the execution of this test function requires calling the target function, the first execution (based on execution) will fail because the target function call is missing. In the second execution, after applying the source code patch, the test framework will correctly call the target function and run as expected. Therefore, if the test framework covers the target function, the results of the two executions should show a "fail-to-pass" transition. We perform execution-based validation for all selected functions to ensure that the HumanEvo test suite

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

TABLE I
DEPENDENCY LEVEL IN HUMANEVO.

Dependency Level	HumanEvo-Python	HumanEvo-Java
Standalone	45 (22.5%)	26 (13.0%)
Intra-class	60 (30.0%)	61 (30.5%)
Inter-class	95 (47.5%)	113 (56.5%)

is reliable.

*3) **Human Annotation**:* We manually rewrite two styles of docstrings for all task instances and label each of them with a dependency level. During the dataset construction process, we observe different docstring styles among these high-quality projects, which can be generally categorized into the detailed and the brief. Detailed docstring typically provides comprehensive descriptions of a function's functionality, elucidates input and output parameter types, and annotates potential error scenarios. Conversely, a brief docstring succinctly outlines the function's functionality in a few sentences. To enhance the diversity of our benchmark and better align with real-world development scenarios, we manually crafted two types of docstring for each function: detailed and brief. This provides users with more options for subsequent usage.

Furthermore, to facilitate the assessment of LLMs' capability in generating functions with varying levels of dependency, we categorize the selected functions based on their dependency levels, including standalone, intra-class, and inter-class. Standalone functions can be implemented without relying on other functions within the project. Intra-class functions require dependencies on other functions within the same class. Inter-class functions imply more complex dependencies, with most of the dependent functions being located in other files within the project. Manual categorization of dependency levels aids in analyzing the performance of LLMs in generating functions of varying complexity, facilitating researchers in improving the application of LLMs in practical development scenarios based on different contexts. Moreover, as indicated in Table I, the majority of our benchmark consists of functions with deep dependency levels. This suggests that our benchmark presents a sufficiently challenging scenario.

Note that we use PRs to construct HumanEvo primarily for the following three reasons. First, since prompting LLMs to generate repository-level functions can be viewed as an incremental development process, and PRs for adding new features exactly mirror the incremental development process in the real world, we choose to instruct LLMs generate the newly added functions in PRs to simulate real-world development scenarios. Second, the quality of functions in PRs is guaranteed. Before merging the newly added code in PRs into the project, repository administrators conduct a review on it, which allows us to select high-quality task instances directly. Third, the base commit in PRs helps us roll back the project to the state before the target function was committed. Finally, by processing PRs appropriately, we can further determine whether the target functions are covered by the project's testing framework.

IV. EXPERIMENTAL DESIGN

In this section, we introduce the models used in the experiments, outline the research questions that the experiments aim to address and provide a detailed description of the experimental setups.

A. Studied LLMs

We select the mainstream LLMs (both open-source and closed-source ones) that have been widely used in recent code generation work [28], [30], [31], [42]. For open-source LLMs, we select the CodeLlama series [11] (including CodeLlama-7B, CodeLlama-13B, and CodeLlama-34B) and the DeepSeekCoder series [13] (including DeepSeekCoder-6.7B and DeepSeekCoder-33B). The CodeLlama and DeepSeekCoder series are popular LLMs that perform well in code generation. For closed-source LLMs, we choose the commonly used commercial models: GPT-3.5-Turbo [36] and GPT-4 [35], with GPT-4 demonstrating excellent performance in previous benchmarks.

B. Studied Context Acquisition Strategies

Due to the large size of modern software repositories, it is not feasible to feed the entire repository to the model. Thus, previous works mainly obtain project context that might be beneficial for generating target functions through two strategies: retrieval-based and static analysis-based methods. In the following experiments, we cover both of these technical routes to demonstrate that the evolution-ignored situation leads to the deviation of the LLMs' performance from reality.

Retrieval-based Context Acquisition. We adopt the standard retrieval-augmented-generation pipeline for repository-level code generation [31], [43]–[45]. In the evolution-aware setting, we roll back the repository to the state before the target code was committed for each task instance. On the contrary, in the evolution-ignored settings, we directly utilize the latest project version as the context source. Then, we construct a corresponding context retrieval database by partitioning the source code files from the repository into a collection of code snippets. Subsequently, we utilize the docstring of the target code as the query for retrieval and compute the Jaccard similarity between the query and all code snippets in the context retrieval database. After sorting all the code snippets by similarity score, we select project contexts with higher similarity scores as project context for target function generation.

Static Analysis-based Context Acquisition. Similar to the static analysis-based repository-level code generation work [46], we obtain code files in the repository that might help generate the target code through static analysis. First, we roll back the repository to the corresponding state according to evolution-ignored or evolution-aware settings. Then, we extract the source code from these files as project context to feed to the model. There are four types of context sources: local file (the file where the target code resides), import file (files imported by the local file in the project), sibling file (files located in the same folder as the local file), and

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

TABLE II
RETRIEVAL-BASED CONTEXT ACQUISITION APPROACH: MODEL PERFORMANCE UNDER EVOLUTION-AWARE (HUMANEVO) AND EVOLUTION-IGNORED (EI) SETTINGS.

Model	Python		Java	
	EI	HumanEvo	EI	HumanEvo
CodeLlama-7B	29.0%	22.0% (↓ 24.1%)	8.5%	6.0% (↓ 29.4%)
CodeLlama-13B	30.5%	23.0% (↓ 24.6%)	9.5%	7.0% (↓ 26.3%)
CodeLlama-34B	31.5%	24.0% (↓ 23.8%)	14.5%	10.0% (↓ 31.0%)
DeepSeekCoder-6.7B	30.0%	23.0% (↓ 23.3%)	13.0%	9.0% (↓ 30.8%)
DeepSeekCoder-33B	31.0%	25.0% (↓ 19.4%)	15.5%	11.5%(↓ 25.8%)
GPT-3.5-Turbo	32.5%	25.5% (↓ 21.5%)	18.0%	13.0% (↓ 27.8%)
GPT-4	34.5%	26.5% (↓ 23.2%)	20.5%	14.5% (↓ 29.3%)

similar file (files in the project with names similar to the local file). We utilize four prompting strategies from [46]: **local** – extracting context solely from the local file; **local+import** – extracting code from both local and import files; **local+sibling** - extracting code from both local and sibling files; **local+sim** - extracting code from both local and similar files.

C. Research Questions

Our experiments intend to answer the following research questions (RQs):

- **RQ1: How effective is HumanEvo in benchmarking the performance of LLMs in repository-level code generation task?**
  We conduct experiments under two different settings, evolution-aware and evolution-ignored, to explore the differences in model performance between these two settings and reveal the actual performance of the models in real development scenarios.
- **RQ2: How does the code generation performance change as repository evolves?**
  We treat multiple versions of project as context sources to explore how the model's performance change as the project evolves.
- **RQ3: How different code descriptions in the dataset affect code generation performance?**
  We conduct experiments to compare the performance of the model under different docstring styles.

D. Experimental Settings

We assess the functionality correctness of code generated by LLMs with execution-based evaluation and utilize the commonly used metric Pass@1 for evaluation. We set the generation temperature of LLMs to 0.8, top-p to 0.95, and the context window length to 4096. Note that, to mitigate issues stemming from the randomness of model generation, the experimental results presented in this paper are obtained by conducting three repeated experiments and averaging the results. The experiment was conducted on a server running Ubuntu 18.04.6 LTS and equipped with 128 Intel(R) Xeon(R) Platinum 8336C @ 2.30GHzE CPUs, and 8 NVIDIA A800 with 80GB memory.

V. EVALUATION RESULTS

A. RQ1: How effective is HumanEvo in benchmarking the performance of LLMs in repository-level code generation task?

Overall Performance. Table II and Table III present the performance of seven mainstream LLMs under two different experimental settings: *Evolution-Ignored* (where the context source is the latest version of the project) and *Evolution-Aware* (HumanEvo, where the context source is the project before the target function was committed). Table II shows the performance of retrieval based project context acquisition approach, and Table III shows the performance of static analysis based project context acquisition approaches. All the experimental results in Table II and Table III show a significant decrease in performance under the evolution-aware setting compared to the evolution-ignored setting. This suggests that ignoring the evolution of the project would lead to inflated performance of LLMs, failing to accurately reflect their real code generation performance.

Break Down Analysis. We further analyze the generation results to investigate whether evolution-ignored setting always leads to over-estimated performance of LLMs. Table IV illustrates the overlap of successful task instances between evolution-aware setting and evolution-ignored setting. Since the results from retrieval-based project context acquisition approach and static analysis based project context acquisition approaches are similar, subsequent analysis primarily focuses on the experimental results of the retrieval-based approach. As introduced above, evolution-ignored setting would raise issues such as the future context leakage issue, which would lead to inflated performance. Meanwhile, evolution-ignored setting can raise issues such as the useful context leakage issue, which would lead to underestimated performance. However, the results in Table II reflect that models' overall performance only exhibits an inflated trend, driving us to delve deeper into whether issues such as useful context leakage has occurred. We investigate the overlap between task instances that LLMs successfully completed under evolution-ignored and evolution-aware settings.

In Table IV, EA stands for the evolution-aware setting and EI stands for the evolution-ignored setting. A ✓means successful, while a × means failure. So, EA✓EI× refers to the number of instances that pass under the evolution-aware setting but fail under the evolution-ignored setting. EA× EI✓represents instances that fail under EA but pass under EI. EA✓EI✓denotes the number of instances that pass under both settings. From the experimental results, it is evident that except for GPT-4 in HumanEvo-Python, which has a value of 0 for EA✓EI× situation, all other LLMs exhibit deviations from reality caused by issues such as the useful context missing issue in a certain quantity, showing that the issues, which are raised by the evolution-ignored setting and would lead to underestimated performance, have impact on evolution-ignored setting. However, due to the significantly higher number in EA× EI✓, the primary influencing factor

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

TABLE III
STATIC ANALYSIS BASED PROJECT CONTEXT ACQUISITION APPROACHES: MODEL PERFORMANCE UNDER EVOLUTION-AWARE (HUMANEVO) AND EVOLUTION-IGNORED (EI) SETTINGS.

Data	Model	local		local+import		local+sibling		local+sim	
		EI	HumanEvo	EI	HumanEvo	EI	HumanEvo	EI	HumanEvo
HumanEvo-Python	CodeLlama-7B	27.0%	24.0%(↓ 11.1%)	28.0%	25.0%(↓ 10.7%)	28.5%	25.0%(↓ 12.3%)	29.0%	25.0%(↓ 13.8%)
	CodeLlama-13B	28.5%	24.5%(↓ 14.0%)	29.5%	26.5%(↓ 10.2%)	29.5%	25.5%(↓ 13.6%)	30.5%	25.5%(↓ 16.4%)
	CodeLlama-34B	29.5%	25.0%(↓ 15.3%)	30.0%	27.0%(↓ 10.0%)	30.0%	26.5%(↓ 11.7%)	30.5%	26.5%(↓ 13.1%)
	DeepSeekCoder-6.7B	29.0%	24.0%(↓ 17.2%)	29.0%	24.5%(↓ 15.5%)	29.0%	25.5%(↓ 12.1%)	29.5%	26.0%(↓ 12.1%)
	DeepSeekCoder-33B	30.5%	25.5%(↓ 16.4%)	31.0%	26.5%(↓ 14.5%)	30.5%	26.5%(↓ 13.1%)	31.0%	26.5%(↓ 14.5%)
	GPT-3.5-Turbo	33.0%	26.5%(↓ 19.7%)	34.0%	27.0%(↓ 20.6%)	32.5%	26.5%(↓ 18.5%)	33.5%	27.5%(↓ 17.9%)
	GPT-4	34.5%	27.0%(↓ 21.7%)	35.0%	28.0%(↓ 20.0%)	34.5%	27.5%(↓ 20.3%)	35.5%	28.0%(↓ 21.1%)
HumanEvo-Java	CodeLlama-7B	16.5%	8.0%(↓ 51.5%)	15.0%	6.5%(↓ 56.7%)	12.0%	6.0%(↓ 50.0%)	13.5%	6.5%(↓ 51.9%)
	CodeLlama-13B	15.0%	6.5%(↓ 56.7%)	16.0%	7.0%(↓ 56.3%)	15.5%	6.5%(↓ 58.1%)	15.5%	6.5%(↓ 58.1%)
	CodeLlama-34B	17.5%	8.0%(↓ 54.3%)	14.5%	7.0%(↓ 51.7%)	12.5%	6.5%(↓ 48.0%)	14.5%	8.0%(↓ 44.8%)
	DeepSeekCoder-6.7B	17.0%	8.5%(↓ 50.0%)	16.5%	10.0%(↓ 39.3%)	14.5%	9.0%(↓ 37.9%)	18.0%	7.0%(↓ 61.1%)
	DeepSeekCoder-33B	16.5%	10.0%(↓ 39.3%)	16.5%	10.0%(↓ 39.3%)	18.5%	8.5%(↓ 54.1%)	19.0%	10.0%(↓ 47.4%)
	GPT-3.5-Turbo	19.5%	11.5%(↓ 41.0%)	21.0%	14.0%(↓ 33.3%)	20.0%	12.0%(↓ 40.0%)	21.5%	13.0%(↓ 39.5%)
	GPT-4	22.0%	13.5%(↓ 38.6%)	23.0%	14.5%(↓ 37.0%)	22.5%	14.0%(↓ 37.8%)	23.5%	14.5%(↓ 38.3%)

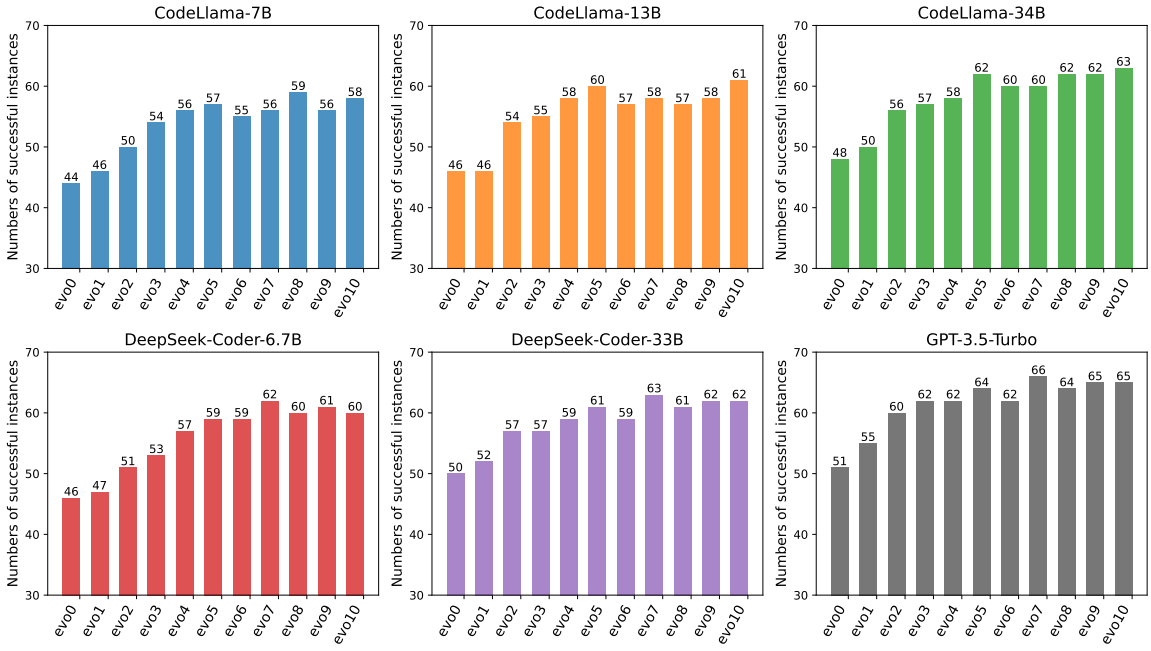


Fig. 5. LLM's performance when using different project versions as context sources.

remains the future context leakage, leading to exaggerated performance of LLMs.

Dependency Level. Functions with different dependency levels vary in their reliance on project context, and the evolution-ignored setting directly affects project context. Therefore, in this experiment, We examine the performance differences of LLMs in generating functions with deeper dependency levels, focusing on non-standalone functions in HumanEvo, which encompass intra-class and inter-class functions, across two settings: evolution-ignored and evolution-aware. As shown in Table V, under the evolution-aware setting, the accuracy of LLM in generating both intra-class and inter-class functions has noticeably declined. The decline is more pronounced for inter-class functions, as these functions typically require cross-file context, which is often more scattered. As the project evolves, its various parts tend to become more comprehensive. Therefore, under the evolution-ignored setting, the models can reference an increasing amount of context. Consequently, when the evolution-aware setting loses access to these contexts, the model's performance significantly deteriorates.

RQ1 Summary: Experiments show an inflated trend in LLM's performance under the evolution-ignored setting.

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

TABLE IV
BREAK DOWN ANALYSIS. EA AND EI STAND FOR EVOLUTION-AWARE AND EVOLUTION-IGNORED, RESPECTIVELY. ✓MEANS SUCCESS, × MEAS FAILURE.

Model	Python			Java		
	EA✓ EI×	EA✓ EI✓	EA× EI✓	EA✓ EI×	EA✓ EI✓	EA× EI✓
CodeLlama-7B	7	37	21	3	9	8
CodeLlama-13B	4	42	19	3	11	8
CodeLlama-34B	2	46	17	6	14	15
DeepSeekCoder-6.7B	4	42	18	7	11	15
DeepSeekCoder-33B	4	46	16	5	18	13
GPT-3.5-Turbo	2	49	16	10	16	20
GPT-4	0	53	15	4	25	16

TABLE V
MODEL PERFORMANCE IN GENERATING FUNCTIONS WITH DEEPER DEPENDENCY LEVELS.

	Model	Python		Java	
		EI	HumanEvo	EI	HumanEvo
Intra-Class	CodeLlama-7B	32.8%	31.1% (↓ 5.0%)	1.6%	1.6%
	CodeLlama-13B	29.5%	27.9% (↓ 5.6%)	2.9%	1.9% (↓ 33.3%)
	CodeLlama-34B	34.4%	27.9% (↓ 19.0%)	8.2%	6.6% (↓ 20.0%)
	DeepSeekCoder-6.7B	34.4%	27.9% (↓ 19.0%)	4.9%	3.3% (↓ 33.3%)
	DeepSeekCoder-33B	37.7%	34.4% (↓ 8.7%)	3.3%	3.3%
	GPT-3.5-Turbo	36.1%	29.5% (↓ 18.2%)	4.9%	4.9%
	GPT-4	37.7%	32.9% (↓ 13.0%)	8.2%	4.9% (↓ 40.0%)
Inter-Class	CodeLlama-7B	27.2%	17.5% (↓ 35.7%)	10.6%	5.3% (↓ 50.0%)
	CodeLlama-13B	31.1%	20.3% (↓ 34.4%)	10.6%	6.2% (↓ 41.7%)
	CodeLlama-34B	30.1%	22.3% (↓ 25.8%)	15.0%	10.6% (↓ 29.4%)
	DeepSeekCoder-6.7B	27.2%	20.4% (↓ 25.0%)	15.9%	11.5% (↓ 27.8%)
	DeepSeekCoder-33B	28.2%	21.4% (↓ 24.1%)	17.7%	13.3% (↓ 25%)
	GPT-3.5-Turbo	31.1%	23.3% (↓ 25.0%)	21.2%	14.2% (↓ 33.3%)
	GPT-4	32.0%	23.3% (↓ 27.3%)	22.1%	15.9% (↓ 28.0%)

B. RQ2: How does the code generation performance change as repository evolves?

To further demonstrate the impact of evolution-ignored setting, we treat different versions of the project as context sources to explore how the performance of LLMs change over time. We start from the version right before the target code was committed and incrementally increase the project versions up to the latest version. Typically, a project's version number consists of two digits in the format "v1.2", where the first digit denotes the major version and the second digit denotes the minor version. In Figure 5, evo0 is the version right before the target code was committed, standing for the evolution-aware setting. For evo1 to evo10, the version numbers are incrementally increased. For example, if evo0 corresponds to the "v1.2" version of the project, then evo1 will be "v1.3".

As shown in Figure 5, we find that the performance of LLMs deviates from the actual performance in the evo0 (evolution-aware) setting, with this deviation increasing as the project evolves. All studied LLMs, regardless of being open-source or closed-source and irrespective of parameter size, consistently exhibit inflated performance and follow a similar trend in performance changes. This result further confirms that ignoring project evolution can lead to unrealistic evaluation of LLMs.

RQ2 Summary: In the evolution-ignored setting, as the project evolves, the performance of LLMs deviates from actual performance, with this deviation increasing over time.

C. RQ3: How different code descriptions in the dataset affect code generation performance?

During the benchmark construction process, we notice that, in real-world development, some projects tend to write detailed docstrings, while the others prefer to describe the function's functionality using only several brief statements. Since these docstrings are used as input to models which may significantly affect model performance, we explore whether the impact of the evolution-ignored setting on model performance varies under different docstring styles. To this end, we manually write both brief and detailed docstrings for each task instance in HumanEvo. In this experiment, we evaluate the performance of the studied LLMs on the two types of manually written docstrings. As shown in Table VI, experimental results indicate that under the evolution-ignored setting, using different styles of docstrings as input consistently leads to inflated performance, and the style of the docstring does not significantly change the impact of the evolution-ignored setting on the inflated performance evaluation. Additionally, the majority of LLMs perform better on detailed docstrings compared to the brief docstrings. We suspect that the detailed docstrings provide a more comprehensive description of the functionality of the target function and include the implementation logic of the target function. In addition, detailed docstrings also provide explanations for the input and output of the target function, enabling LLMs to better understand the tasks to be addressed. Conversely, brief docstrings only introduce the intention of the target function, which may lead to LLMs struggling to understand user intent in certain complex scenarios, ultimately resulting in decreased performance.

However, the decline in LLMs' performance is not significant. We suspect that the reason could possibly be related to our use of a retrieval-augmented generation approach, by which we have retrieved a substantial amount of context from the project through similarity calculations. These contexts contribute to a lengthy prompt. However, when LLMs deal with overly long prompts, their attention gets dispersed, leading to the neglect of some useful information by the model.

RQ3 Summary: From the experimental results, we do not observe any correlation between docstring styles and evolution settings. Compared to brief docstrings, detailed docstrings provide more beneficial information to LLMs, resulting in better performance. However, the performance improvement on detailed docstrings is not substantial.

VI. THREATS TO VALIDITY

We have identified the following potential threats that may affect the validity of our study. Firstly, due to limited computational resources, our experiments were not able to cover all the available code LLMs, thus not fully reflecting the performance of all LLMs in actual development scenarios, which may

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

TABLE VI
MODEL PERFORMANCE UNDER DIFFERENT DOCSTRINGS STYLES.

Model	Docstring	Python		Java	
		EI	HumanEvo	EI	HumanEvo
CodeLlama-7B	Detailed	29.0%	22.0%	8.5%	6.0%
	Brief	28.5%	22.0%	7.0%	4.5%
CodeLlama-13B	Detailed	30.5%	23.0%	9.5%	7.0%
	Brief	30.0%	22.5%	8.5%	5.0%
CodeLlama-34B	Detailed	31.5%	24.0%	14.5%	10.0%
	Brief	31.0%	24.5%	11.5%	7.0%
DeepSeekCoder-6.7B	Detailed	30.0%	23.0%	13.0%	9.0%
	Brief	30.0%	23.0%	10.5%	6.5%
DeepSeekCoder-33B	Detailed	31.0%	25.0%	15.5%	11.5%
	Brief	31.0%	24.0%	13.5%	9.0%
GPT-3.5-Turbo	Detailed	32.5%	25.5%	18.0%	13.0%
	Brief	31.5%	24.5%	14.5%	11.0%
GPT-4	Detailed	34.5%	26.5%	20.5%	14.5%
	Brief	32.5%	25.0%	16.5%	12.5%

slightly affect the representativeness of our experiments. Secondly, currently HumanEvo only includes two programming languages. In the future, we plan to address this limitation by continuously expanding HumanEvo to cover as many programming languages as possible, thereby providing a more comprehensive evaluation. Thirdly, in prompt format selection, we ultimately chose the one that yielded the best results after trying several formats. However, since we did not cover all available prompt formats, the one we selected may not be the optimal one. Lastly, our benchmark includes GitHub data before 2023, which might already be present in the pretraining dataset of the models we studied, which poses a risk of data leakage. To mitigate this risk, we have implemented an automated benchmark construction pipeline that continuously updates the dataset and rewrited the docstrings for every task instance in HumanEvo.

VII. RELATED WORK

A. Code Generation Benchmarks

Previously, the vast majority of benchmarks were constructed to evaluate the performance of LLMs in generating standalone functions [**?**], [47]–[55], with the most representative ones being HumanEval [56] and MBPP [57]. HumanEval comprises 164 manually crafted programming problems covering language comprehension, reasoning, algorithms, and simple mathematics. MBPP consists of 974 short Python function problems primarily designed for novice programmers.

Finding previous benchmarks fail to reflect complex programming scenarios, researchers propose CoderEval [28], the first repository-level code generation benchmark, to evaluate the performance of existing LLMs in real-world development by prompting them to generate functions selected in actual complex software. CoderEval meticulously selects 230 Python and 230 Java programming tasks from popular real-world open-source projects, evaluating the functional correctness of generated code based on execution. EvoCodeBench [30] collects data from 25 real-world repositories. It also ensures that the distribution of programming tasks (with or without dependencies) matches the distribution of real repositories. RepoCoder [31] proposed RepoEval, which consists of the latest and high-quality real-world repositories covering scenarios such as line completion, API invocation, and function body completion. Currently, most benchmarks for LLM code generation provide simple functions, mostly targeting general functions rather than specific domains, thus limiting practicality. However, BioCoder [58], a benchmark for bioinformatics code generation, has been introduced. BioCoder carefully selects 28 high-quality projects related to bioinformatics, then parses these repositories to generate ASTs and obtain relevant data. A custom code context is written for each filtered function, including necessary imports, cross-file dependencies, and some test cases.

However, all the benchmarks mentioned above have a common flaw: they all overlook the evolution of projects over time. These benchmarks take the latest version of the project as the context source, and such an evolution-ignored setting makes it difficult for them to accurately reflect the performance of LLMs in real-world development scenarios. Therefore, in this work, we propose HumanEvo, the first repository-level evolution-aware code generation benchmark. Before obtaining the project context for the target function, we roll back the repository to the state before the target function was committed to ensure that the project context provided to LLMs does not suffer from issues such as future context leakage and missing useful context. Our objective is to offer LLMs a more realistic development scenario, thus more accurately reflecting the actual performance of LLMs in real-world development scenarios.

B. Repository-level Code Generation

Many technical studies have been proposed to better tackle repository-level code generation tasks [31], [42], [59]–[69]. DocPrompting [60] uses natural language (NL) intent to retrieve documents related to the target code and then sends NL intent and retrieved documents to LLMs. CoCoMIC [70] develops a cross-file context finder, which is a static code analysis tool used to retrieve the most related cross-file context. MGD [71] leverages IDEs to assist in providing context. IDE is invoked to provide useful suggestions, when reaching the pre-defined trigger point. RLPG [46] analyses dependencies potentially existing in the target function, then extracts project context that may be conducive to generating the target function. RepoCoder [31] first slices the source code files in the repository to construct a retrieval library, then uses the generated problem as a query, calculates similarity to obtain project context, and feeds the project context and problem to the model to generate initial results. After obtaining the initial results, the previous generation results are used as queries for further querying to obtain more context. This process is repeated to continuously optimize the generation results. A^3-CodGen [72] first constructs a repository knowledge base covering knowledge of third-party libraries and extracts information on mobile and code context. During the code generation process, LLM first generates an initial version of

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

code based on NL intent, then uses NL intent and the initial version of code to retrieve as much foundational knowledge as possible from the knowledge base for the LLM. Finally, local, global, and third-party library information is integrated into the prompt to allow the model to generate the final results.

All the mentioned works treat the latest version of the project as the context source, falling into the category of evolution-ignored scenarios. Although in RepoCoder, researchers begin to take notice of this issue by considering all code after the target function in the corresponding file as "future context", this still falls short of reality and introduces a lack of the context that should be present. Therefore, we propose a novel repository-level code generation benchmark, HumanEvo, an evolution-aware pragmatic benchmark, to better simulate real development scenarios for LLMs.

C. LLMs for Code Generation

CodeLlama [11] is further trained based on Llama2 [73]. It is capable of understanding and generating code in various programming languages, including but not limited to Python, Java, JavaScript, etc. CodeLlama not only generates complete code but also provides suggestions and optimization solutions for specific code snippets, greatly assisting developers. During the training of the base model, weights are initialized with an equivalent parameter amount of the Llama2 model, followed by training on a dataset of 500 billion tokens of code. StarCoder [12] is derived from StarCoderBase, which is trained using 10 trillion tokens from The Stack, a vast collection of licensed GitHub repositories with inspection tools and selection exit procedures. The DeepSeek-Coder [13] takes into account the often intricate dependency relationships inherent in real-world projects. The model needs to fully consider these dependencies before completing programming tasks. Therefore, to equip DeepSeek-Coder to handle real-world complex projects, the training data is structured to support repository-level comprehension, enhancing the model's understanding of complex projects. This enables the model to utilize cross-file context effectively. MetaGPT [7] constructs a multi-agent collaborative framework, employing Standardized Operating Procedures and involving multiple roles such as programmers and product managers to collaborate on project completion. The results generated by agents are not saved in conversational form but in structured output formats such as charts, files, etc. Communication between agents is also conducted through structured files. MetaGPT also implements an iterative development pipeline, optimizing results through multiple rounds of iteration by recording information generated during the development process by different roles. Similarly, ChatDev [8] adopts a multi-agent framework, allocating and assigning different AI agents to various job functions. ChatDev facilitates communication among different role agents, forming a complete software development process where each agent performs its respective tasks while collaborating uniformly. In addition to these models, there are many other excellent models used in the field of code generation [17]–[23].

VIII. Conclusion

In this paper, we highlight a previously overlooked flaw in evaluating existing repository-level code generation methods: the evolution of projects over time. To address it, we introduce HumanEvo, an evolution-aware repository code generation benchmark, and conduct a comprehensive empirical study to explore LLMs' performance in real-world development scenarios. Experimental results demonstrate that the evolution-ignored situation leads to the inflated performance of LLMs. By incorporating temporal considerations into the benchmark, HumanEvo provides a more comprehensive assessment of model performance in pragmatic development scenarios. We believe that our empirical results and HumanEvo will offer valuable insights for the future development of LLMs-based repository-level code generation methods and benchmarks.

IX. Acknowledgements

This work is supported by the National Natural Science Foundation of China (62032025), the Guangdong Basic and Applied Basic Research Foundation (2023A1515012292) and CCF-Huawei Populus Grove Fund CCFHuaweiSE202403.

References

[1] Z. Zheng, K. Ning, Y. Wang, J. Zhang, D. Zheng, M. Ye, and J. Chen, "A survey of large language models for code: Evolution, benchmarking, and future trends," *arXiv preprint arXiv:2311.10372*, 2023.

[2] Z. Zheng, K. Ning, J. Chen, Y. Wang, W. Chen, L. Guo, and W. Wang, "Towards an understanding of large language models in software engineering tasks," *arXiv preprint arXiv:2308.11396*, 2023.

[3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[4] Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-collaboration code generation via chatgpt," *arXiv e-prints*, pp. arXiv–2304, 2023.

[5] S. Jiang, Y. Wang, and Y. Wang, "Selfevolve: A code evolution framework via large language models," *arXiv preprint arXiv:2306.02907*, 2023.

[6] S. Zhang, Z. Chen, Y. Shen, M. Ding, J. B. Tenenbaum, and C. Gan, "Planning with large language models for code generation," in *The Eleventh International Conference on Learning Representations*, 2023.

[7] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations*, 2024.

[8] C. Qian, X. Cong, W. Liu, C. Yang, W. Chen, Y. Su, Y. Dang, J. Li, J. Xu, D. Li, Z. Liu, and M. Sun, "Communicative agents for software development," 2023.

[9] Y. Wang, T. Jiang, M. Liu, J. Chen, and Z. Zheng, "Beyond functional correctness: Investigating coding style inconsistencies in large language models," *arXiv preprint arXiv:2407.00456*, 2024.

[10] Y. Wang, Y. Huang, D. Guo, H. Zhang, and Z. Zheng, "Sparsecoder: Identifier-aware sparse transformer for file-level code summarization," *arXiv preprint arXiv:2401.14727*, 2024.

[11] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for code," 2024.

[12] R. L. et al., "Starcoder: may the source be with you!" *Transactions on Machine Learning Research*, 2023, reproducibility Certification.

[13] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang, "Deepseek-coder: When the large language model meets programming – the rise of code intelligence," 2024.

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

[14] Q. Zheng, X. Xia, X. Zou, Y. Dong, S. Wang, Y. Xue, L. Shen, Z. Wang, A. Wang, Y. Li, T. Su, Z. Yang, and J. Tang, "Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x," ser. KDD '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 5673–5684. [Online]. Available: https://doi.org/10.1145/3580305.3599790

[15] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, "Palm: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.

[16] L. B. Allal, R. Li, D. Kocetkov, C. Mou, C. Akiki, C. M. Ferrandis, N. Muennighoff, M. Mishra, A. Gu, M. Dey *et al.*, "Santacoder: don't reach for the stars!" *arXiv preprint arXiv:2301.03988*, 2023.

[17] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "Wizardcoder: Empowering code large language models with evol-instruct," in *The Twelfth International Conference on Learning Representations*, 2024.

[18] F. Christopoulou, G. Lampouras, M. Gritta, G. Zhang, Y. Guo, Z. Li, Q. Zhang, M. Xiao, B. Shen, L. Li, H. Yu, L. Yan, P. Zhou, X. Wang, Y. Ma, I. Iacobacci, Y. Wang, G. Liang, J. Wei, X. Jiang, Q. Wang, and Q. Liu, "Pangu-coder: Program synthesis with function-level language modeling," 2022.

[19] B. Shen, J. Zhang, T. Chen, D. Zan, B. Geng, A. Fu, M. Zeng, A. Yu, J. Ji, J. Zhao, Y. Guo, and Q. Wang, "Pangu-coder2: Boosting large language models for code with ranking feedback," 2023.

[20] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," in *The Eleventh International Conference on Learning Representations*, 2023.

[21] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[22] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. Sutherland Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-level code generation with alphacode," p. 1092–1097, Dec. 2022.

[23] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, S. Yih, L. Zettlemoyer, and M. Lewis, "Incoder: A generative model for code infilling and synthesis," in *The Eleventh International Conference on Learning Representations*, 2023.

[24] L. Nie, J. Sun, Y. Wang, L. Du, S. Han, D. Zhang, L. Hou, J. Li, and J. Zhai, "Unveiling the black box of plms with semantic anchors: towards interpretable neural semantic parsing," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 13 400–13 408.

[25] W. Gu, Y. Wang, L. Du, H. Zhang, S. Han, D. Zhang, and M. Lyu, "Accelerating code search with deep hashing and code classification," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 2534–2544.

[26] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, "Magicoder: Source code is all you need," in *International Conference on Machine Learning*, 2023.

[27] Y. Wang, Y. Wang, D. Guo, J. Chen, R. Zhang, Y. Ma, and Z. Zheng, "Rlcoder: Reinforcement learning for repository-level code completion," 2024.

[28] H. Yu, B. Shen, D. Ran, J. Zhang, Q. Zhang, Y. Ma, G. Liang, Y. Li, Q. Wang, and T. Xie, "Codereval: A benchmark of pragmatic code generation with generative pre-trained models," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–12.

[29] T. Liu, C. Xu, and J. McAuley, "Repobench: Benchmarking repository-level code auto-completion systems," in *The Twelfth International Conference on Learning Representations*, 2024.

[30] J. Li, G. Li, X. Zhang, Y. Dong, and Z. Jin, "Evocodebench: An evolving code generation benchmark aligned with real-world code repositories," 2024.

[31] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "Repocoder: Repository-level code completion through iterative retrieval and generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2471–2484.

[32] L. Voinea, A. Telea, and J. J. van Wijk, "Cvsscan: visualization of code evolution," in *Proceedings of the 2005 ACM Symposium on Software Visualization*, ser. SoftVis '05. New York, NY, USA: Association for Computing Machinery, 2005, p. 47–56. [Online]. Available: https://doi.org/10.1145/1056018.1056025

[33] D. Cleland, "The evolution of project management," *IEEE Transactions on Engineering Management*, vol. 51, no. 4, pp. 396–397, 2004.

[34] G. Robles, J. Amor, J. Gonzalez-Barahona, and I. Herraiz, "Evolution and growth in large libre software projects," in *IWPSE*, 2005, pp. 165–174.

[35] OpenAI, "Gpt-4 technical report," 2024.

[36] J. Ye, X. Chen, N. Xu, C. Zu, Z. Shao, S. Liu, Y. Cui, Z. Zhou, C. Gong, Y. Shen, J. Zhou, S. Chen, T. Gui, Q. Zhang, and X. Huang, "A comprehensive capability analysis of gpt-3 and gpt-3.5 series models," 2023. [Online]. Available: https://arxiv.org/abs/2303.10420

[37] B. Negoita, G. Vial, M. Shaikh, and A. Labbe, "Code forking and software development project sustainability: Evidence from github," 2019.

[38] L. Dabbish, C. Stuart, J. Tsay, and J. Herbsleb, "Social coding in github: transparency and collaboration in an open software repository," in *Proceedings of the ACM 2012 conference on computer supported cooperative work*, 2012, pp. 1277–1286.

[39] E. Kalliamvakou, D. Damian, K. Blincoe, L. Singer, and D. M. German, "Open source-style collaborative development practices in commercial projects using github," in *2015 IEEE/ACM 37th IEEE international conference on software engineering*, vol. 1. IEEE, 2015, pp. 574–585.

[40] R. Kallis, A. Di Sorbo, G. Canfora, and S. Panichella, "Predicting issue types on github," *Science of Computer Programming*, vol. 205, p. 102598, 2021.

[41] M. M. Rahman and C. K. Roy, "An insight into the pull requests of github," in *Proceedings of the 11th working conference on mining software repositories*, 2014, pp. 364–367.

[42] H. N. Phan, H. N. Phan, T. N. Nguyen, and N. D. Q. Bui, "Repohyper: Better context retrieval is all you need for repository-level code completion," 2024.

[43] T. Formal, C. Lassance, B. Piwowarski, and S. Clinchant, "From distillation to hard negative sampling: Making sparse neural ir models more effective," in *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*, 2022, pp. 2353–2359.

[44] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "ColBERTv2: Effective and efficient retrieval via lightweight late interaction," in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, M. Carpuat, M.-C. de Marneffe, and I. V. Meza Ruiz, Eds. Seattle, United States: Association for Computational Linguistics, Jul. 2022, pp. 3715–3734. [Online]. Available: https://aclanthology.org/2022.naacl-main.272

[45] L. Xiong, C. Xiong, Y. Li, K.-F. Tang, J. Liu, P. N. Bennett, J. Ahmed, and A. Overwijk, "Approximate nearest neighbor negative contrastive learning for dense text retrieval," in *International Conference on Learning Representations*, 2021.

[46] D. Shrivastava, H. Larochelle, and D. Tarlow, "Repository-level prompt generation for large language models of code," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 693–31 715.

[47] B. Athiwaratkun, S. K. Gouda, Z. Wang, X. Li, Y. Tian, M. Tan, W. U. Ahmad, S. Wang, Q. Sun, M. Shang, S. K. Gonugondla, H. Ding, V. Kumar, N. Fulton, A. Farahani, S. Jain, R. Giaquinto, H. Qian, M. K. Ramanathan, R. Nallapati, B. Ray, P. Bhatia, S. Sengupta, D. Roth, and B. Xiang, "Multi-lingual evaluation of code generation models," in *The Eleventh International Conference on Learning Representations*, 2023.

[48] F. Cassano, J. Gouwar, D. Nguyen, S. Nguyen, L. Phipps-Costin, D. Pinckney, M.-H. Yee, Y. Zi, C. J. Anderson, M. Q. Feldman *et al.*, "A scalable and extensible approach to benchmarking nl2code for 18 programming languages," *arXiv preprint arXiv:2208.08227*, 2022.

[49] N. Jain, S. Vaidyanath, A. Iyer, N. Natarajan, S. Parthasarathy, S. Rajamani, and R. Sharma, "Jigsaw: Large language models meet program synthesis," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1219–1231.

[50] D. Zan, B. Chen, D. Yang, Z. Lin, M. Kim, B. Guan, Y. Wang, W. Chen, and J.-G. Lou, "Cert: Continual pre-training on sketches for library-oriented code generation," in *Proceedings of the Thirty-First*

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio

International Joint Conference on Artificial Intelligence, IJCAI-22, L. D. Raedt, Ed. International Joint Conferences on Artificial Intelligence Organization, 7 2022, pp. 2369–2375, main Track. [Online]. Available: https://doi.org/10.24963/ijcai.2022/329

[51] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, "Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation," 2023.

[52] Y. Ding, Z. Wang, W. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth *et al.*, "Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[53] M. A. M. Khan, M. S. Bari, X. L. Do, W. Wang, M. R. Parvez, and S. Joty, "xcodeeval: A large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval," 2023.

[54] J. Chen, Q. Zhong, Y. Wang, K. Ning, Y. Liu, Z. Xu, Z. Zhao, T. Chen, and Z. Zheng, "Rmcbench: Benchmarking large language models' resistance to malicious code," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 995–1006.

[55] Z. Zhang, Y. Wang, C. Wang, J. Chen, and Z. Zheng, "Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation," *arXiv preprint arXiv:2409.20550*, 2024.

[56] M. C. et al., "Evaluating large language models trained on code," 2021.

[57] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[58] X. Tang, B. Qian, R. Gao, J. Chen, X. Chen, and M. Gerstein, "Biocoder: A benchmark for bioinformatics code generation with contextual pragmatic knowledge," 2024.

[59] D. Shrivastava, D. Kocetkov, H. de Vries, D. Bahdanau, and T. Scholak, "Repofusion: Training code models to understand your repository," 2024.

[60] S. Zhou, U. Alon, F. F. Xu, Z. Jiang, and G. Neubig, "Docprompting: Generating code by retrieving the docs," in *The Eleventh International Conference on Learning Representations*, 2023.

[61] D. Zan, B. Chen, Y. Gong, J. Cao, F. Zhang, B. Wu, B. Guan, Y. Yin, and Y. Wang, "Private-library-oriented code generation with large language models," *arXiv preprint arXiv:2307.15370*, 2023.

[62] D. Zan, B. Chen, Z. Lin, B. Guan, W. Yongji, and J.-G. Lou, "When language model meets private library," in *Findings of the Association for Computational Linguistics: EMNLP 2022*, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 277–288. [Online]. Available: https://aclanthology.org/2022.findings-emnlp.21

[63] R. Bairi, A. Sonwane, A. Kanade, A. Iyer, S. Parthasarathy, S. Rajamani, B. Ashok, S. Shet *et al.*, "Codeplan: Repository-level coding using llms and planning," *arXiv preprint arXiv:2309.12499*, 2023.

[64] M. Liang, X. Xie, G. Zhang, X. Zheng, P. Di, wei jiang, H. Chen, C. Wang, and G. Fan, "Repofuse: Repository-level code completion with fused dual context," 2024.

[65] A. Eghbali and M. Pradel, "De-hallucinator: Iterative grounding for llm-based code completion," 2024.

[66] K. Zhang, J. Li, G. Li, X. Shi, and Z. Jin, "Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges," 2024.

[67] C. Wang, J. Zhang, Y. Feng, T. Li, W. Sun, Y. Liu, and X. Peng, "Teaching code llms to use autocompletion tools in repository-level code generation," 2024.

[68] L. Guo, Y. Wang, E. Shi, W. Zhong, H. Zhang, J. Chen, R. Zhang, Y. Ma, and Z. Zheng, "When to stop? towards efficient code generation in llms with excess token prevention," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1073–1085.

[69] J. Chen, C. Chen, J. Hu, J. Grundy, Y. Wang, T. Chen, and Z. Zheng, "Identifying smart contract security issues in code snippets from stack overflow," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1198–1210.

[70] Y. Ding, Z. Wang, W. Ahmad, M. Ramanathan, R. Nallapati, P. Bhatia, D. Roth, and B. Xiang, "Cocomic: Code completion by jointly modeling in-file and cross-file context, december 2022," *URL http://arxiv.org/abs/2212.10007*.

[71] L. A. Agrawal, A. Kanade, N. Goyal, S. Lahiri, and S. Rajamani, "Monitor-guided decoding of code lms with static analysis of repository context," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[72] D. Liao, S. Pan, X. Sun, X. Ren, Q. Huang, Z. Xing, H. Jin, and Q. Li, "a^3-codgen: A repository-level code generation framework for code reuse with local-aware, global-aware, and third-party-library-aware," 2024.

[73] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open foundation and fine-tuned chat models," 2023.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 31,2026 at 10:42:08 UTC from IEEE Xplore. Restrictio